Lecture 28.09: Decision Tree Classifier in Python (Titanic Dataset)

📌 Topic Overview

In this hands-on coding lecture, we implement a Decision Tree Classifier using Scikit-Learn on the famous **Titanic dataset** (`sns.load_dataset('titanic')`), inspect missing values, encode features, train a baseline model, and observe severe overfitting.

🚢 Dataset & Preprocessing Pipeline

The instructor loads the Titanic dataset from Seaborn and selects the core features:

- **Features (X)**: `pclass` , `sex` , `fare` , `embarked` , `age`
- **Target (y)**: `survived` (1 = Survived, 0 = Did not survive)

1. Handling Missing Data with `SimpleImputer`

- `age` : Imputed with the **median** value (`SimpleImputer(strategy="median")`).
- `embarked` : Imputed with the **most frequent** value (`SimpleImputer(strategy="most_frequent")`).

2. Encoding Categorical Features

- `sex` : Encoded to numerical (e.g., female=0, male=1).
- `embarked` : Encoded to numerical (e.g., C=0, Q=1, S=2).

⚠️ Training the Baseline Model: The Overfitting Trap

When we train an unconstrained `DecisionTreeClassifier(random_state=42)` :

- **Training Accuracy**: $\approx 98.2\%$ (Almost perfect memorization!)
- **Testing Accuracy**: $\approx 77.5\%$

The huge gap between training and testing performance clearly shows that an unpruned tree memorizes training data quirks rather than learning generalizable patterns.

💻 Complete Python Code from Lecture

```
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# 1. Load Titanic dataset
titanic = sns.load_dataset('titanic')

# 2. Impute missing values
imp_median = SimpleImputer(strategy="median")
titanic["age"] = imp_median.fit_transform(titanic[["age"]])

imp_freq = SimpleImputer(strategy="most_frequent")
titanic["embarked"] = imp_freq.fit_transform(titanic[["embarked"]]).ravel()

# 3. Simple Encoding
titanic["sex"] = titanic["sex"].map({"female": 0, "male": 1})
titanic["embarked"] = titanic["embarked"].map({"C": 0, "Q": 1, "S": 2})

# 4. Select features and target
X = titanic[["pclass", "sex", "fare", "embarked", "age"]]
y = titanic["survived"]

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

# 6. Fit baseline unpruned tree
dt_base = DecisionTreeClassifier(random_state=42)
dt_base.fit(X_train, y_train)

# 7. Evaluate
train_acc = dt_base.score(X_train, y_train)
test_acc = dt_base.score(X_test, y_test)

print(f"Training Accuracy: {train_acc:.4f}")
print(f"Testing Accuracy:  {test_acc:.4f}")
print(f"Overfitting Gap:   {(train_acc - test_acc):.4f}")
```

🔑 Key Takeaways

1. Real-world datasets require proper missing value handling (`SimpleImputer`) and categorical encoding before fitting trees.
2. An unconstrained decision tree achieves near 100% training accuracy but drops significantly on test data.
3. Tree visualization (`plot_tree`) reveals an overly deep, complex tree that requires pruning.